A Project Report on

# Satya-Score: An AI Hallucination Detection & Scoring Framework

*Submitted in partial fulfilment of the requirements for the degree of*
**Bachelor of Technology**
in Artificial Intelligence and Data Science


*Submitted by*

Vivek Yeshwantrao Borkar
Piyush Devidas Nande
Aasawari Sahebrao Jumade
Arunima Tridip Wankhade

*Under the guidance of*

**Dr. R. R. Karwa**

**Department of Artificial Intelligence and Data Science**
**Prof. Ram Meghe Institute of Technology & Research, Badnera, Amravati**

**2026-27**


© 2026 Vivek Yeshwantrao Borkar, Piyush Devidas Nande, Aasawari Sahebrao Jumade, Arunima Tridip Wankhade. All Rights Reserved.

# CERTIFICATE

This is to certify that the project report entitled "Satya-Score: An AI Hallucination Detection & Scoring Framework" is a bonafide work carried out by

Vivek Yeshwantrao Borkar
Piyush Devidas Nande
Aasawari Sahebrao Jumade
Arunima Tridip Wankhade

in partial fulfilment of the requirements for the degree of Bachelor of Technology in Artificial Intelligence and Data Science of Prof. Ram Meghe Institute of Technology & Research, Badnera, Amravati, during the academic year 2026-27, under my guidance.

**Dr. R. R. Karwa**
**Project Guide**

**[Head of Department name]**
**Head of Department**

**[Principal name]**
**Principal**

Place: Badnera, Amravati
Date: _____

# DECLARATION

We declare that this report, "Satya-Score: An AI Hallucination Detection & Scoring Framework", is our own original work, carried out under the guidance of Dr. R. R. Karwa in the Department of Artificial Intelligence and Data Science, Prof. Ram Meghe Institute of Technology & Research, Badnera, Amravati. It has not been copied from any published or unpublished source; the work of others is acknowledged through citations. We are the authors of this work and the owners of its copyright.

| Sr. No. | Name of the Student | Signature | Date |
|---|---|---|---|
| 1 | Vivek Yeshwantrao Borkar | | |
| 2 | Piyush Devidas Nande | | |
| 3 | Aasawari Sahebrao Jumade | | |
| 4 | Arunima Tridip Wankhade | | |

# ACKNOWLEDGEMENT

We express our sincere gratitude to our project guide, Dr. R. R. Karwa, for steady guidance, critical questions and encouragement at every stage of this project.

We thank the Head of the Department of Artificial Intelligence and Data Science and the Principal of Prof. Ram Meghe Institute of Technology & Research, Badnera, Amravati for the facilities and support that made this work possible, and all teaching and non-teaching staff of the department for their help.

We are grateful to the people who build and maintain the open resources this work relies on: Wikipedia, Wikidata, World Bank Open Data, the WHO Global Health Observatory, the fact-checking organisations whose ratings are published through the Google Fact Check Tools API, and the authors of the open-source models and libraries listed in this report. Finally, we thank our families and friends for their constant support.

Vivek Yeshwantrao Borkar
Piyush Devidas Nande
Aasawari Sahebrao Jumade
Arunima Tridip Wankhade

# ABSTRACT

Chatbots based on large language models write fluent answers that sometimes contain false statements, and a reader has no easy way to tell which sentence is wrong; for Hindi, checking tools are scarce. Satya-Score is a framework that detects and scores such hallucinations in English and Hindi text at the level of single claims. An answer is divided into self-contained claims. A claim that official statistics (World Bank, WHO) or the Wikidata knowledge graph can settle is decided by that source; otherwise a matching published fact-check is used; failing both, Wikipedia passages are judged by a multilingual natural-language-inference model under a conservative evidence rule. Each claim receives a score, a verdict (Verified, Uncertain or Hallucinated) and its evidence, together with the correct fact whenever a structured source decided it; the whole text receives an overall verdict. On 128 unseen true sentences the system raised 4 false alarms. The statistics check flagged all 137 altered figures without contradicting a true one, and fact-check matching made no wrong decision. The knowledge graph was right in 11 of its 13 decisions in the pre-registered run, and in all 12 after a linking error was corrected in an exploratory re-run. The system is cautious: most hallucinations that no structured source covers are missed. The report also records the changes that failed their tests and the decisions that overrode a test, among them a language-model judge that caught more errors but nearly quadrupled false alarms.

**Keywords:** hallucination detection, fact verification, natural language inference, knowledge graph, Hindi, large language models

# TABLE OF CONTENTS


| Sr. No. | Title | Page No. |
|---|---|---|
| | **Certificate** | |
| | **Declaration** | |
| | **Acknowledgement** | |
| | **Abstract** | |
| | **List of Tables** | |
| | **List of Figures** | |
| | **List of Screenshots** | |
| **CHAPTER 1: INTRODUCTION** | | **1** |
| 1.1 | Background: Hallucination in Large Language Models | 1 |
| 1.2 | Motivation | 1 |
| 1.3 | Problem Statement | 1 |
| 1.4 | Objectives | 2 |
| 1.5 | Scope of the Project | 2 |
| 1.6 | Organisation of the Report | 2 |
| **CHAPTER 2: LITERATURE REVIEW** | | **3** |
| 2.1 | Fact Verification and Claim Checking | 3 |
| 2.2 | Hallucination Detection in LLM Output | 3 |
| 2.3 | Grounding Checkers | 3 |
| 2.4 | Multilingual and Hindi Fact-Checking | 4 |
| 2.5 | Knowledge Graphs and Cross-Lingual Retrieval | 4 |
| 2.6 | Summary of the Literature | 4 |
| 2.7 | Research Gap | 5 |
| **CHAPTER 3: SYSTEM ARCHITECTURE/ANALYSIS** | | **6** |
| 3.1 | Existing System and its Limitations | 6 |
| 3.2 | Proposed System | 6 |
| 3.3 | Requirement Analysis | 7 |
| 3.4 | Hardware and Software Requirements | 7 |
| 3.5 | Feasibility Study | 8 |
| 3.6 | Overall System Architecture | 8 |
| **CHAPTER 4: SYSTEM DESIGNS** | | **10** |
| 4.1 | Block Diagram | 10 |
| 4.2 | Data Flow Diagrams | 11 |
| 4.3 | Module Design | 12 |
| 4.4 | Algorithms | 13 |
| 4.5 | Database Design | 16 |
| 4.6 | Flowchart of a Fact-Check Request | 17 |
| **CHAPTER 5: IMPLEMENTATION & RESULTS** | | **18** |
| 5.1 | Technologies Used | 18 |
| 5.2 | Implementation of the Modules | 18 |

| Sr. No. | Title | Page No. |
|---|---|---|
| 5.3 | Web Application | 18 |
| 5.4 | Datasets and Annotation | 21 |
| 5.5 | Evaluation Method | 22 |
| 5.6 | Results on Real Chatbot Claims and Unseen Data | 23 |
| 5.7 | Results of the Structured Sources | 23 |
| 5.8 | Changes Tested and Not Adopted | 24 |
| 5.9 | Performance | 25 |
| 5.10 | Discussion and Limitations | 25 |
| **CHAPTER 6: CONCLUSION** | | **27** |
| 6.1 | Conclusion | 27 |
| 6.2 | Future Scope | 27 |
| | **References** | 28 |
| | **List of Published Papers and IPR Filed** | 32 |
| | **Resume** | 33 |

# LIST OF TABLES


| Table No. | Title | Page No. |
|---|---|---|
| 2.1 | Comparison of existing systems | 5 |
| 3.1 | Hardware requirements | 7 |
| 3.2 | Software requirements | 8 |
| 4.1 | Main modules | 12 |
| 4.2 | Database tables | 16 |
| 5.1 | Technologies used | 18 |
| 5.2 | Datasets | 21 |
| 5.3 | Results of the running system on chatbot claims and unseen data | 23 |
| 5.4 | Results of the structured sources | 23 |
| 5.5 | Changes tested and the decisions taken | 24 |
| 5.6 | Performance before and after optimisation (laptop GPU) | 25 |

# LIST OF FIGURES


| Figure No. | Title | Page No. |
|---|---|---|
| 3.1 | Overall system architecture | 9 |
| 4.1 | Block diagram of the pipeline | 10 |
| 4.2 | Data flow diagram, level 0 | 11 |
| 4.3 | Data flow diagram, level 1 | 12 |
| 4.4 | Flowchart of a fact-check request | 17 |
| Listing 4.1 | Evidence rule | 14 |
| Listing 4.2 | Two-agency agreement for statistics | 14 |
| Listing 4.3 | Direction-aware fact-check matching | 15 |
| Listing 4.4 | Claim scoring | 16 |

# LIST OF SCREENSHOTS


| Screenshot No. | Title | Page No. |
|---|---|---|
| 5.1 | Input page with an example answer | 19 |
| 5.2 | Results page: whole-text verdict and highlighted claims | 19 |
| 5.3 | Claim cards with the correct fact from Wikidata | 20 |
| 5.4 | A Hindi claim decided by a published fact-check | 20 |
| 5.5 | History of saved checks (account e-mail hidden) | 21 |

# CHAPTER 1
# INTRODUCTION

## 1.1 Background: Hallucination in Large Language Models

Chatbots built on large language models (LLMs), for example GPT-4 [1] and Llama [2], now answer everyday questions on health, public services, science and history. They produce text by repeatedly choosing a likely next word; they do not consult a record of facts while writing. As a result, an answer can be grammatical, confident and still contain statements that are invented or wrong. Such output is usually called hallucination [3], [4]. Studies of long model-written answers have found that a noticeable share of their individual facts is not supported by any reliable source [5].

Satya-Score ("satya" means truth) is a framework for finding these errors. It breaks an answer into individual factual claims, checks each claim against the most authoritative source available, and reports for every claim a verdict, a score and the evidence behind it.

## 1.2 Motivation

A wrong date, name or figure inside a long and otherwise correct answer is easy to miss, and checking every sentence by hand removes the benefit of using an assistant. Automatic checkers exist, but most work only for English, give a single score for a whole passage, or depend on another large model whose own judgement may be wrong. Hindi is spoken by hundreds of millions of people and appears more and more in chatbot output, yet a Hindi reader has almost no tool that points to the false statement in an answer and explains why it is false. This gap motivated the project.

## 1.3 Problem Statement

Given an answer produced by an LLM in English or Hindi, identify its factual claims; decide for each claim whether reliable sources support it, contradict it or cannot settle it; show the evidence and, where a structured source holds it, the correct fact; and do this quickly enough to be useful while reading, without marking true statements as false.

### 1.4 Objectives

i) To analyse hallucinations in LLM answers.

ii) To review existing detection methods.

iii) To design a bilingual claim-level detection framework.

iv) To develop a secure real-time web application.

v) To evaluate the system on labelled and unseen data.

### 1.5 Scope of the Project

- **Languages:** English and Hindi text, typed, pasted or uploaded as a .txt or .pdf file.
- **Claims:** statements of fact. Questions, greetings, advice and opinions are recognised and not checked.
- **Sources:** English and Hindi Wikipedia, Wikidata, World Bank Open Data, the WHO Global Health Observatory and published fact-checks found through the Google Fact Check Tools API.
- **Output:** a verdict, score and explanation for every claim, an overall verdict for the text, and a history of checks for signed-in users.
- **Outside the scope:** other languages, images and audio, correcting the chatbot itself, and commercial use (the WHO data licence allows non-commercial use only).

### 1.6 Organisation of the Report

Chapter 2 reviews related work and identifies the research gap. Chapter 3 analyses the existing approaches and the requirements and presents the overall architecture. Chapter 4 gives the design: block diagram, data flow diagrams, modules, algorithms, database and flowchart. Chapter 5 describes the implementation and reports the results, including the changes that were tested and rejected and the limitations. Chapter 6 concludes and outlines future work.

# CHAPTER 2
# LITERATURE REVIEW

## 2.1 Fact Verification and Claim Checking

Automatic fact verification became a standard task with FEVER [6], in which a system receives a claim, finds evidence in Wikipedia and labels the claim as supported, refuted or lacking enough information. Later work asked whether a generated text is consistent with a given source. The TRUE study [7] compared many such consistency metrics on a common footing, and AlignScore [8] trained a single alignment function over many tasks for the same purpose. These methods assume that the right evidence is already available; finding it is a separate problem.

## 2.2 Hallucination Detection in LLM Output

Benchmarks such as TruthfulQA [9] and HaluEval [10] measure how often models produce false or unsupported statements. FActScore [5] splits a generated biography into atomic facts and checks each against Wikipedia, and SAFE [11] uses an LLM agent that searches the web for every fact. Both show that checking individual facts is more informative than scoring a whole answer, but both are built for English and rely on large models for splitting or judging. SelfCheckGPT [12] needs no external source: it samples several answers and treats disagreement between them as a warning, which cannot catch an error the model repeats consistently. Other work operates at a finer grain: RefChecker [13] checks claim triplets against a reference, ANAH [14] supplies sentence-level analytical annotations for training hallucination annotators, FAVA [15] detects and edits erroneous spans, and MIND [16] uses a model's internal states. A known weakness of inference models is that they can rely on word overlap rather than meaning [17]; this weakness reappears in the error analysis of Chapter 5.

## 2.3 Grounding Checkers

A grounding checker decides whether a sentence is supported by a supplied document. MiniCheck [18] reaches accuracy close to GPT-4 on this task at a much smaller size and cost, and AlignScore [8] was the strongest small checker before it. Retrieval-augmented generation [19] takes a related route, giving the model retrieved

documents while it writes; it can reduce errors in a system one controls, but it cannot audit text that a third-party chatbot has already produced.

### 2.4 Multilingual and Hindi Fact-Checking

The SemEval-2025 Mu-SHROOM shared task [20] posed span-level hallucination detection in 14 languages, Hindi among them. Participating systems used, for example, an ensemble of LLMs that vote on candidate spans [21], natural-language inference with a model fine-tuned on synthetic data [22], and semantic-role decomposition with retrieved context [23]; none of these three uses a knowledge graph. SHROOM-CAP [24] extended the task to scientific publications in nine languages, and BHRAM-IL [25] provides a benchmark that measures hallucination in Hindi, Gujarati, Marathi and Odia as well as English. Multilingual encoders such as XLM-R [26] and MuRIL [27], the IndicNLPSuite resources [28], IndicSBERT [29] and IndicXNLI [30] supply building blocks for Indian languages, while inference models are commonly trained on MultiNLI [31] and its cross-lingual extension XNLI [32].

### 2.5 Knowledge Graphs and Cross-Lingual Retrieval

A knowledge graph stores facts as explicit relations between entities. Wikidata [33] is open, multilingual and edited collaboratively. FactKG [34] verifies claims by reasoning over graph paths, and KGR [35] uses a graph to retrofit and correct a model's draft answer. MultiHal [36] is a multilingual benchmark that pairs questions with knowledge-graph paths in several European languages and reports that answers grounded in those paths score better on similarity and inference measures. Two recent surveys conclude that the benefit of knowledge graphs against hallucination is promising but not yet settled [37], [38]. For retrieval, sentence encoders such as Sentence-BERT [39] and its multilingual distillation [40] map text into vectors that can be searched quickly with FAISS [41]; DeBERTaV3 [42] is a strong encoder family on which multilingual inference models are built.

### 2.6 Summary of the Literature

Table 2.1 summarises representative systems along the properties that matter for this project.

**Table 2.1: Comparison of existing systems**

| System | Unit checked | Hindi | Evidence used | Gives correct fact |
|---|---|---|---|---|
| FActScore [5] | Atomic facts | No | Wikipedia passages | No |
| SAFE [11] | Atomic facts | No | Web search through an LLM agent | No |
| SelfCheckGPT [12] | Sentences | No | None (agreement among sampled answers) | No |
| MiniCheck [18] | Sentences | No | A supplied document | No |
| Mu-SHROOM systems [21], [22], [23] | Text spans | Yes | LLMs, inference models or retrieved context; no knowledge graph | No |
| MultiHal [36] | Benchmark (question answering) | No | Knowledge-graph paths | Not applicable |
| BHRAM-IL [25] | Benchmark | Yes | Not applicable | Not applicable |
| Satya-Score (this work) | Claims | Yes | Official statistics, Wikidata, published fact-checks, Wikipedia | Yes, when statistics or Wikidata decide |

## 2.7 Research Gap

Claim-level checking, Hindi coverage and knowledge-graph grounding have each been studied, but in separate lines of work. The multilingual detectors reviewed here do not use structured sources, and the graph-grounded work does not cover Hindi. None of the reviewed systems routes each claim to official statistics, a knowledge graph or a published fact-check before falling back on text, states the correct fact when it finds an error, and does this for Hindi in near real time. Satya-Score addresses this combination. Its components are existing public models and data sources; the contribution lies in how they are combined, in the safeguards placed on each source, and in an evaluation that reports failures as well as successes.

# CHAPTER 3
# SYSTEM ARCHITECTURE/ANALYSIS

## 3.1 Existing System and its Limitations

Today a reader who doubts a chatbot answer has three options: check each statement by hand, look for an article by a professional fact-checking organisation, or use one of the research tools reviewed in Chapter 2. Each has limits:

- Manual checking is slow and is rarely done for every sentence.
- Professional fact-checks cover popular rumours, not the particular facts in a particular answer.
- Most research tools handle English only, return one score for a whole text, or ask another LLM for the verdict, which can itself be wrong.
- Few tools explain a verdict with a source the reader can open, and fewer still state what the correct fact is.

## 3.2 Proposed System

Satya-Score checks each claim separately and sends it to the strongest source able to settle it. Official statistics and the Wikidata knowledge graph are consulted first, because they hold exact values. If neither answers, a published fact-check of the same claim is used when one exists. Otherwise, passages from English and Hindi Wikipedia are retrieved and judged by a multilingual inference model under a conservative rule that prefers “Uncertain” to a false alarm. Every verdict carries its evidence, and an error decided by a structured source is shown together with the correct value.

### 3.3 Requirement Analysis

**Functional requirements**

FR1. Accept English or Hindi text, typed or pasted or uploaded as .txt or .pdf, with the language chosen by the user or detected from the script.

FR2. Extract self-contained factual claims and skip questions, greetings, advice and opinions.

FR3. Verify each claim against official statistics, Wikidata, published fact-checks and Wikipedia.

FR4. Give each claim a score and a verdict (Verified, Uncertain or Hallucinated), with its source, its evidence and, when available, the correct fact.

FR5. Give the whole text an overall verdict and score.

FR6. Allow users to register, sign in, keep a history of checks and export results as CSV or JSON.

**Non-functional requirements**

- **Reliability:** true statements should rarely be marked Hallucinated; when the evidence is weak the system should answer Uncertain.
- **Performance:** a typical answer should be checked within seconds.
- **Security:** passwords and sessions must be protected and requests from other websites refused.
- **Transparency:** every verdict must name its source.
- **Portability:** the system must run on a laptop, with or without a GPU.

### 3.4 Hardware and Software Requirements

Table 3.1 lists the hardware on which the system was developed and measured, and Table 3.2 the software. The memory figures were measured with the server running on the CPU.

**Table 3.1: Hardware requirements**

| Component | Used for development | Needed to run the server |
|---|---|---|
| Processor | Intel Core i5-13450HX | A 64-bit processor (CPU-only operation works, more slowly) |
| Graphics | NVIDIA GeForce RTX 4050 Laptop GPU, 6 GB | Optional; speeds up the models |

| Component | Used for development | Needed to run the server |
|---|---|---|
| Memory | 24 GB RAM | About 4 GB (measured: 2.3 GB after loading, 2.9 GB at peak) |
| Network | Internet connection | Internet connection for the live look-ups |

**Table 3.2: Software requirements**

| Purpose | Software (version used) |
|---|---|
| Operating system | Windows 11 (development); Linux container image for cloud hosting |
| Back end | Python 3.13, FastAPI 0.115.6, Uvicorn 0.34.0, SQLite (Python standard library) |
| Models and search | PyTorch 2.13.0, Transformers 5.14.1, Sentence-Transformers 5.6.0, FAISS (faiss-cpu 1.14.3) |
| Language processing | spaCy 3.8.14 with en_core_web_sm, Indic NLP Library 0.92, pdfminer.six |
| Front end | React 18, TypeScript 5, Vite 5, Tailwind CSS 3, built with Node.js 24 |
| Client | Any modern web browser |

### 3.5 Feasibility Study

- **Technical:** all models are open and run on a laptop GPU or CPU; the structured sources are reached through public web interfaces.
- **Economic:** the models, libraries and data sources are free of charge; the Google Fact Check Tools API needs only an API key.
- **Operational:** users need only a web browser; results are shown in place, with links to the sources.
- **Legal:** the models are released under the MIT or Apache-2.0 licences; Wikidata is CC0, World Bank data CC BY 4.0, Wikipedia text CC BY-SA 4.0, and WHO data CC BY-NC-SA 3.0 IGO, which limits the system to non-commercial use unless the WHO source is switched off.

### 3.6 Overall System Architecture

Fig. 3.1 shows the architecture. The React web application sends the text to the FastAPI back end, which handles accounts and sessions and stores history in SQLite. The back end extracts the claims, consults the four kinds of source for each claim, and

returns the scored and explained result. All figures in this report were drawn by the authors.

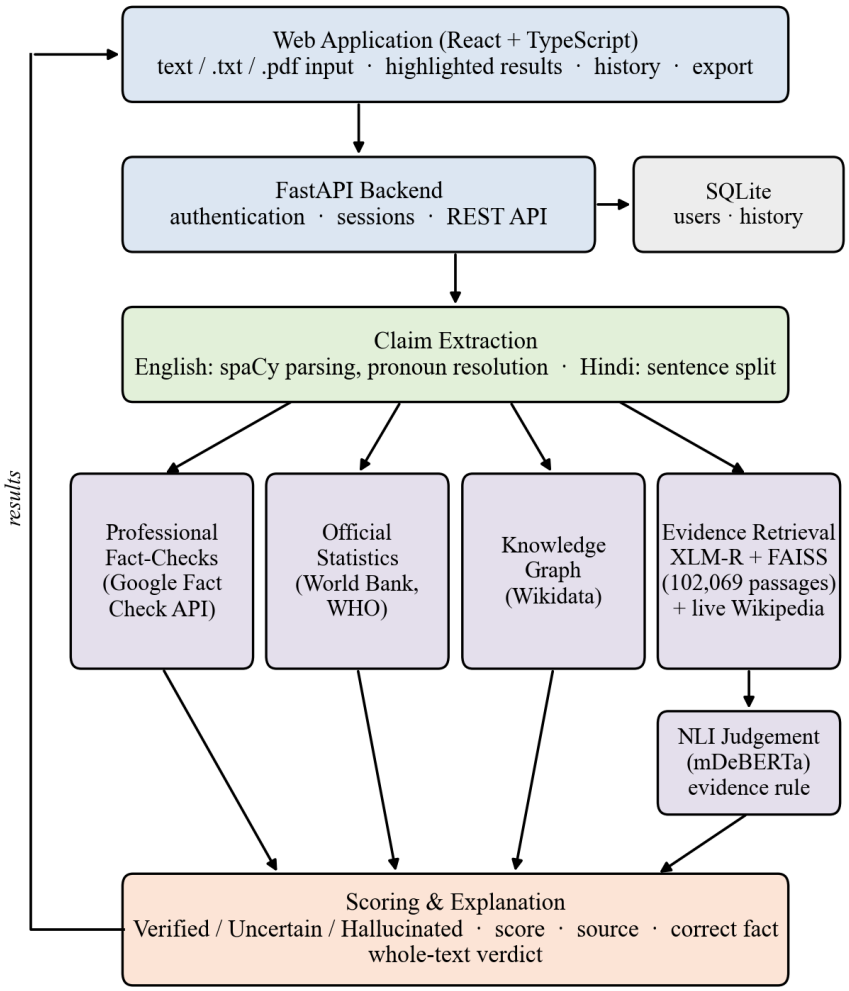



**Fig. 3.1: Overall system architecture**

# CHAPTER 4
# SYSTEM DESIGNS

## 4.1 Block Diagram

Fig. 4.1 shows the processing blocks of a check and, in particular, the order in which the sources decide a claim.

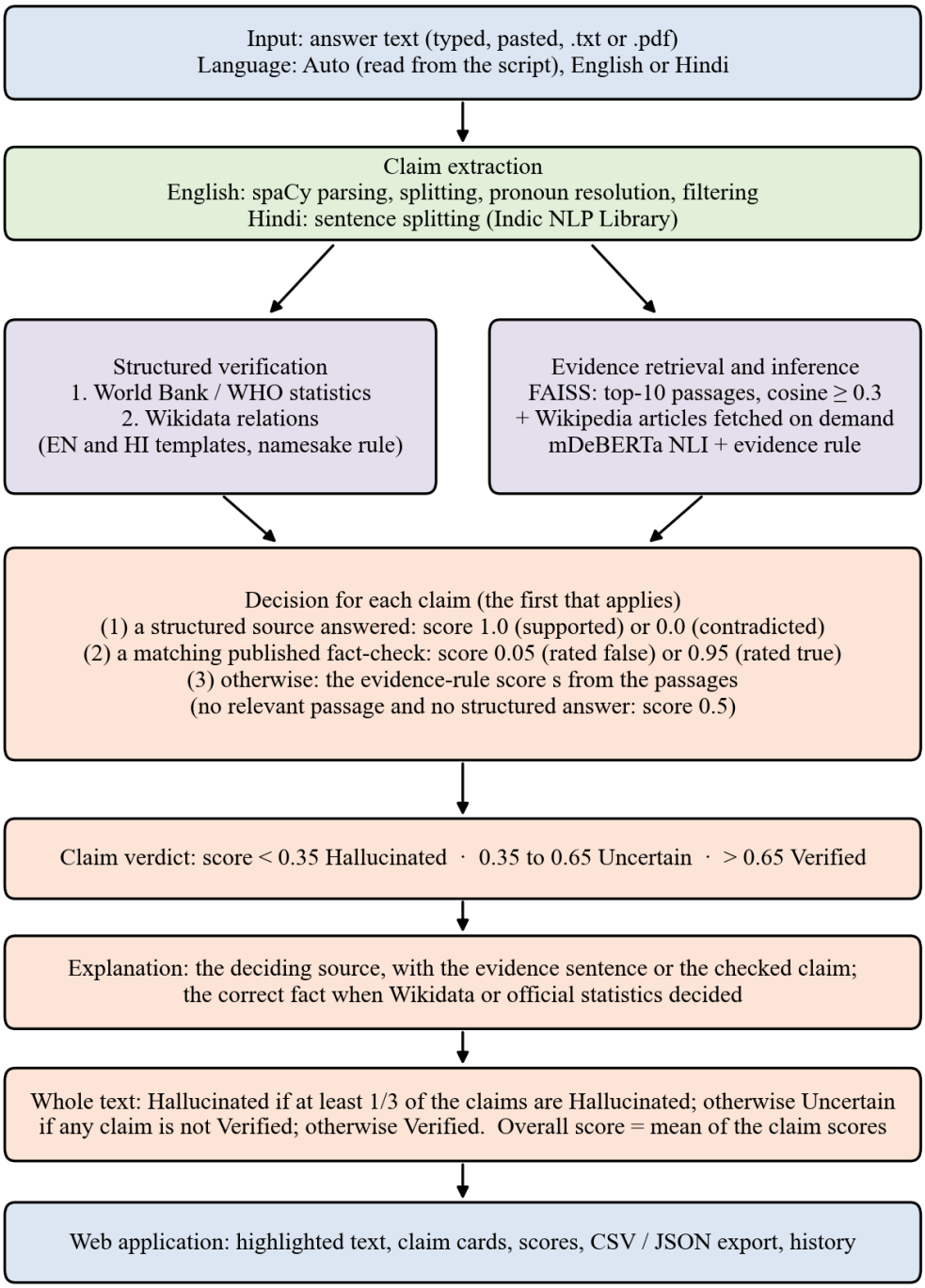



**Fig. 4.1: Block diagram of the pipeline**

### 4.2 Data Flow Diagrams

Fig. 4.2 is the context-level (level 0) data flow diagram: answer text comes in from the user, evidence comes from four external sources, and checks are stored in the user database. Fig. 4.3 refines the system into six processes. Processes 2.0, 3.0 and 4.0 work on the claims produced by process 1.0; process 5.0 combines their results, and process 6.0 keeps accounts and history.

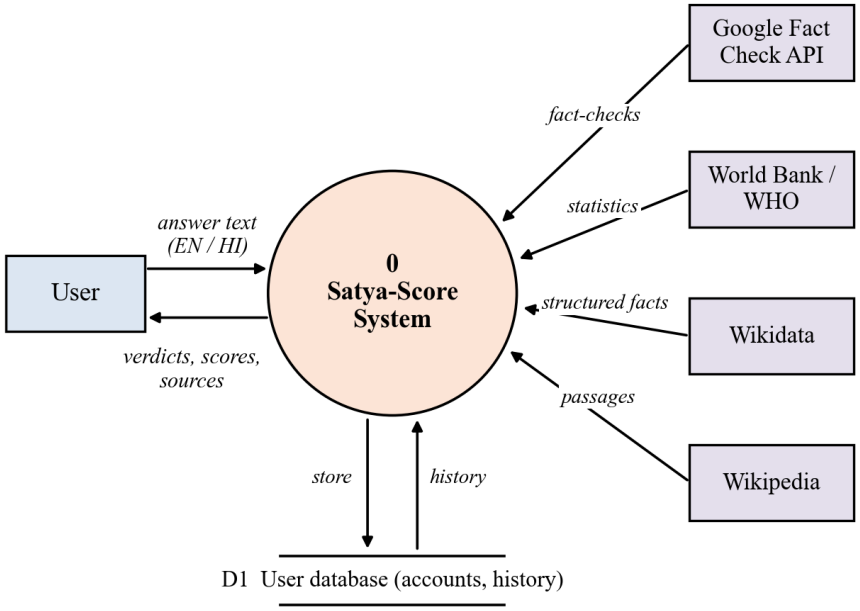



**Fig. 4.2: Data flow diagram, level 0**

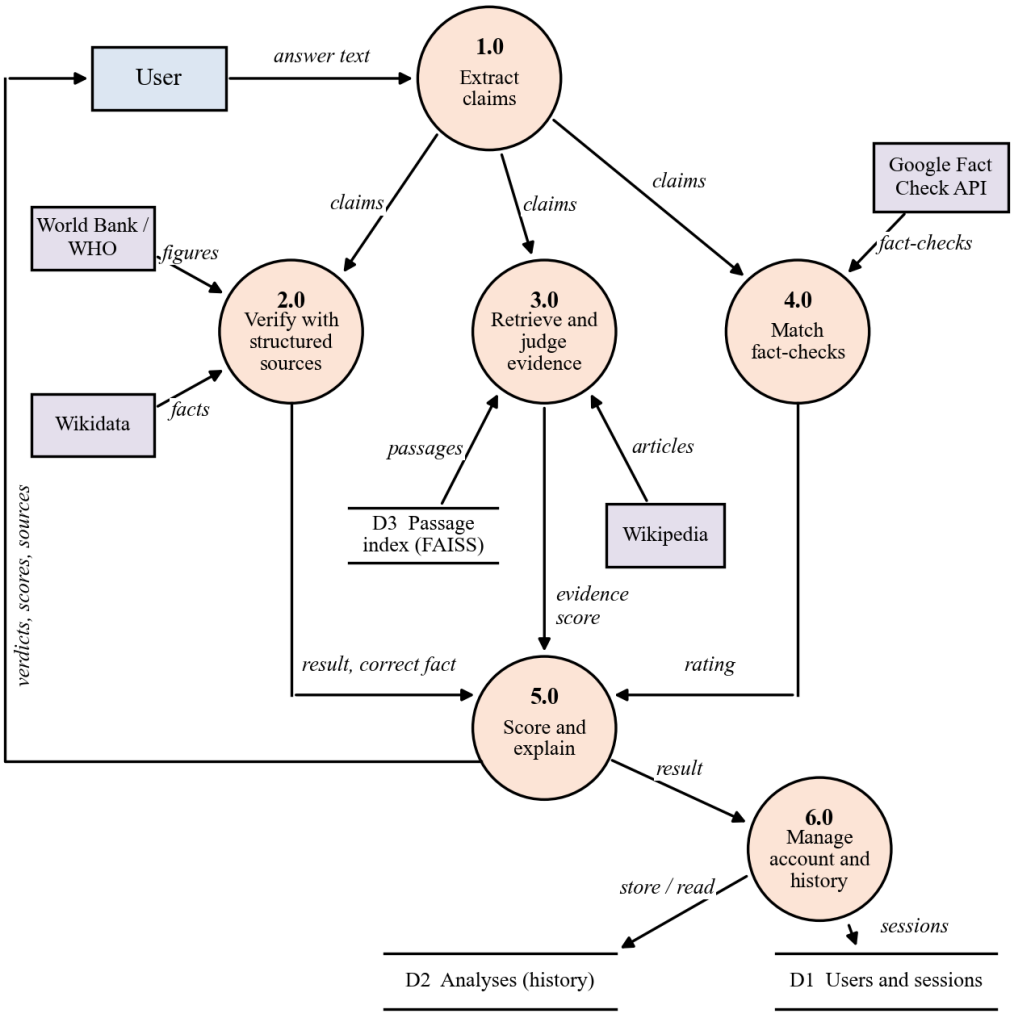



**Fig. 4.3: Data flow diagram, level 1**

## 4.3 Module Design

The code is organised in three Python packages (models, data sources, back end) and a React application. Table 4.1 lists the main modules.

**Table 4.1: Main modules**

| Module | Responsibility |
|---|---|
| member3_backend/claim_extractor.py | Claim extraction, pronoun resolution, filtering of non-factual sentences |
| member3_backend/pipeline.py | Orchestration: retrieval, inference, structured checks, fact-checks, scoring, explanations; parallel look-ups |
| member3_backend/evidence_rules.py | Evidence rule and on-topic test |
| member3_backend/factcheck_rules.py | Direction-aware matching of published fact-checks |
| member2_data/entity_linker.py, wikidata_connector.py, kg_extra.py | Entity linking (English and Hindi) and Wikidata checks |

| Module | Responsibility |
|---|---|
| member2_data/official_stats.py | World Bank and WHO statistics with two-agency agreement |
| member1_model/ | Embeddings, inference scorer, CMHSA scoring and weight training |
| member3_backend/api.py, auth.py, serve.py | Web API, accounts and security, production serving |
| web/src/ | React and TypeScript web application |

## 4.4 Algorithms

### 4.4.1 Claim extraction

English text is parsed with spaCy [43]. Coordinated statements are split into separate claims and list items become claims of their own. A pronoun is replaced by the topic of the passage rather than by the nearest name; on 400 held-out Wikipedia introductions this raised the number of correctly resolved pronouns from 93 to 277 in the pre-registered test. Questions, greetings, advice and opinions are recognised and dropped. Hindi text is split into sentences with the Indic NLP Library [44], and questions, greetings and advice are dropped.

### 4.4.2 Evidence retrieval

Claims and passages are encoded with a multilingual XLM-R sentence encoder [40], [26] and compared in a FAISS index [41] built from 102,069 Wikipedia passages (89,594 English and 12,475 Hindi). When a claim concerns an article that is not yet indexed, the system fetches the article from Wikipedia, embeds it and adds it to the index while it runs. The ten most similar passages are retrieved, and those with a cosine similarity of at least 0.3 are passed on.

### 4.4.3 Inference and the evidence rule

A multilingual mDeBERTa-v3 model [42] fine-tuned on MultiNLI and XNLI [31], [32] gives, for each passage p, the probability e(p) that it entails the claim and the probability c(p) that it contradicts it. The passages are combined by the evidence rule:

$$s = 0.5 + 0.5 \cdot \max e(p) \qquad \text{if } \max e(p) \geq 0.5$$

$$s = 0.5 + 0.5 \cdot (\max e(p) - \max c(q) \text{ over on-topic } q) \qquad \text{otherwise}$$

A passage is on-topic when the title of its article, or a form of that title, appears in the claim. Support from any retrieved passage can therefore verify a claim, but only a

passage about the claim's own subject can contradict it. The rule removes the most common false alarm of the earlier version, in which a passage about a different subject contradicted the wording of the claim. It also removes many correct detections; Section 5.8 reports this cost.

**Listing 4.1: Evidence rule (member3_backend/evidence_rules.py)**

```
def aggregate(rows):
    """rows: (P(entail), P(contradict), on_topic) for each passage."""
    if not rows:
        return 0.5
    max_e = max(e for e, _, _ in rows)
    if max_e >= ENTAIL_WINS:                    # 0.5: support from any passage
        return 0.5 + 0.5 * max_e
    on_topic_c = [c for _, c, t in rows if t]    # only on-topic passages contradict
    return 0.5 + 0.5 * (max_e - (max(on_topic_c) if on_topic_c else 0.0))
```

### 4.4.4 Structured verification

- **Official statistics.** Claims about the population, GDP, life expectancy, adult literacy or infant mortality of a country are compared with World Bank [45] figures and, for life expectancy and infant mortality, also with WHO [46] figures. A claim is judged at the precision it is written to: a GDP written as "US$ 0.1 billion" is not contradicted by an official 0.066 billion. When both agencies report a value they must agree on the verdict; if they disagree, no verdict is given.
- **Knowledge graph.** Claims about places and dates of birth and death, founding dates, capitals, authorship, current office-holders, quantities and currencies are matched to Wikidata [33] through English and Hindi relation templates. When several entities share the claimed name, one of them is enough to support the claim, but a contradiction requires every namesake that has a recorded value to disagree.

**Listing 4.2: Two-agency agreement for statistics (member2_data/official_stats.py)**

```
def _agrees(claimed, official, tol, absolute, half=0.0):
    # within the tolerance, or within the precision the claim is written to
    return abs(claimed - official) <= max(tol if absolute else tol * official, half)


votes = {_agrees(claimed, v, tol, absolute, half) for _, _, v, _ in figures}
if len(votes) > 1:                 # World Bank and WHO disagree: no verdict
    return _unknown(kind, claimed)
status = "supported" if votes == {True} else "contradicted"
```

### 4.4.5 Matching published fact-checks

Fact-checks are found through the Google Fact Check Tools API [47]. A rating of "false" is carried over only if our claim entails the checked claim, and a rating of "true" only if the checked claim entails ours; neither is carried over when either

direction reads as a contradiction or when the two claims are not similar enough (cosine below 0.5). This keeps a negated claim, such as “hot water does not kill the coronavirus”, from inheriting the rating of its opposite. If the applicable fact-checks disagree, none is used.

**Listing 4.3: Direction-aware fact-check matching (member3_backend/factcheck_rules.py)**

```
@property
def applies(self):
    """Does the fact-check's clear rating carry over to our claim?"""
    if self.similarity < FC_MIN_SIM or self.contra > FC_MAX_CONTRA:   # 0.50 / 0.20
        return False
    if self.check.verdict == "false":
        return self.entail_claim_to_check >= FC_MIN_ENTAIL            # 0.70
    if self.check.verdict == "true":
        return self.entail_check_to_claim >= FC_MIN_ENTAIL
    return False
```

### 4.4.6 Scoring: the Composite Multilingual Hallucination Scoring Algorithm (CMHSA)

CMHSA was designed as a weighted combination of three signals for a claim c with evidence E and knowledge graph G:

$$HalluScore(c) = \alpha \cdot SemSim(c, E) + \beta \cdot NLI(c, E) + \gamma \cdot KG(c, G), \quad \alpha + \beta + \gamma = 1$$

SemSim is the similarity of the best retrieved passage, NLI the evidence-rule score s, and KG the knowledge-graph answer. The weights are produced by a softmax, so they are non-negative and sum to one, and they are trained separately for each language with class-balanced weighting on the training claims. A signal that is missing for a claim is left out and the remaining weights are rescaled. Two design decisions, both made during development, determine how the formula behaves in practice:

- **Similarity is excluded:** SemSim measures whether a passage is on the same topic, not whether it supports the claim, so it was excluded from training and α is fixed at exactly 0. The trained weights are β = 0.500 and γ = 0.500 for English and β = 0.498 and γ = 0.502 for Hindi. Similarity is still used, but only as the 0.3 relevance cut-off that decides which passages reach the inference model.
- **The knowledge graph decides when it answers:** if statistics or Wikidata return supported or contradicted, the score is 1.0 or 0.0 and the other signals are not mixed in. This rule was chosen from an exploratory analysis of the 40 relational claims (Section 5.7).

Since α = 0 and a knowledge-graph answer overrides the formula, the weighted sum never actually blends two signals in the running system. The score of a claim is therefore set by the first of the following that applies:

1. Statistics or Wikidata answered: score 1.0 if supported, 0.0 if contradicted; the correct value is shown.
2. A published fact-check applies: score 0.05 if rated false, 0.95 if rated true.
3. Relevant passages were found: the evidence-rule score s.
4. None of these: score 0.5 (insufficient evidence).

The verdict follows from the score: below 0.35 Hallucinated, above 0.65 Verified, and Uncertain in between.

**Listing 4.4: Claim scoring (member1_model/cmhsa.py)**

```
alpha, beta, gamma = self.get_weights(language)        # alpha = 0.0 (SemSim excluded)
raw = {"semsim": (alpha, sub_scores.sem_sim), "nli": (beta, sub_scores.nli_prob),
       "kg": (gamma, sub_scores.kg_conf)}
active = {k: (w, v) for k, (w, v) in raw.items() if v is not None}
if KG_DECIDES and "kg" in active:                       # statistics / Wikidata answered
    hallu_score = float(sub_scores.kg_conf)             # 1.0 supported, 0.0 contradicted
else:
    weight_sum = sum(w for w, _ in active.values())
    hallu_score = sum((w / weight_sum) * v for w, v in active.values())
```

### 4.4.7 Whole-text verdict

The overall verdict is taken from the claim verdicts, not from an average, so that one false claim cannot be hidden among many true ones. A text is Hallucinated when at least one third of its claims are Hallucinated; otherwise it is Uncertain if any claim is Hallucinated or Uncertain; otherwise it is Verified. The overall score shown beside the verdict is the mean of the claim scores.

## 4.5 Database Design

Accounts and history are stored in SQLite. Table 4.2 lists the tables. Passwords are stored only as salted scrypt hashes, and session tokens and password-reset codes only as SHA-256 hashes, so a copy of the database cannot be used to sign in.

**Table 4.2: Database tables**

| Table | Main columns | Purpose |
|---|---|---|
| users | id, email (unique), name, password_hash, created_at | Registered accounts |
| sessions | token_hash, user_id, created_at, expires_at | Signed-in sessions |

| Table | Main columns | Purpose |
|---|---|---|
| analyses | id, user_id, created_at, language, input_text, result_json, total_claims, flagged_count | History of checks |
| password_resets | code_hash, user_id, expires_at | One-time password-reset codes (valid 30 minutes) |

### 4.6 Flowchart of a Fact-Check Request

Fig. 4.4 traces a single request from the input text to the final verdict.

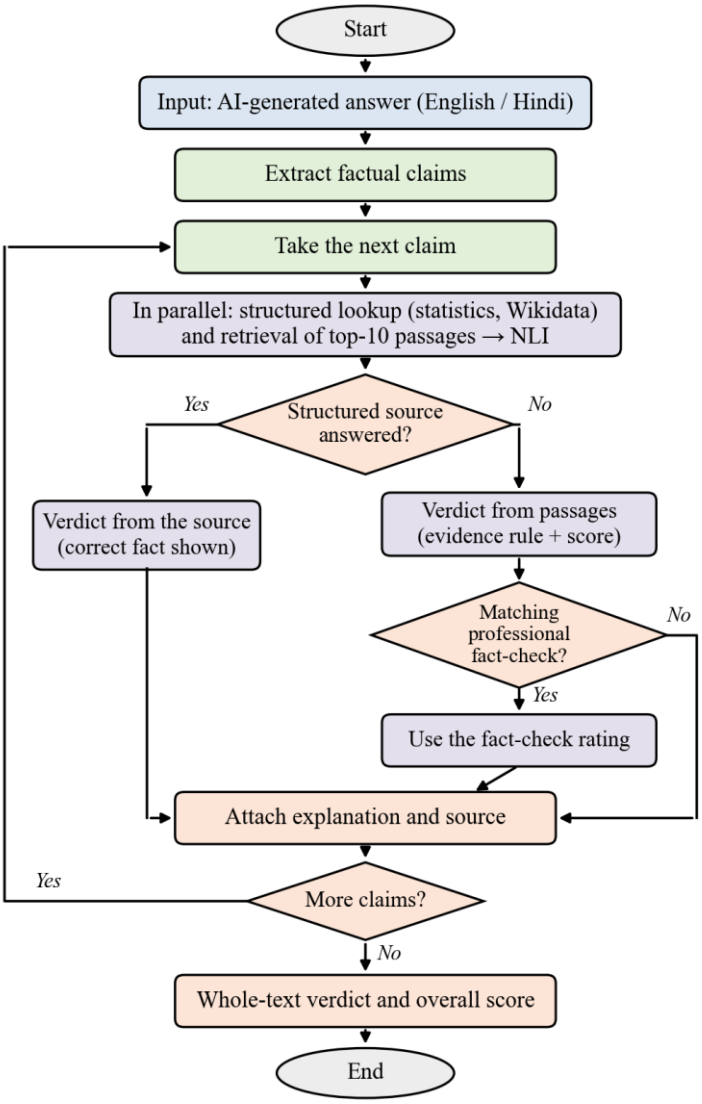



**Fig. 4.4: Flowchart of a fact-check request**

# CHAPTER 5
# IMPLEMENTATION & RESULTS

## 5.1 Technologies Used

**Table 5.1: Technologies used**

| Purpose | Technology |
|---|---|
| Programming | Python 3.13 (back end), TypeScript (web application) |
| Deep learning | PyTorch [48], Hugging Face Transformers [49], Sentence-Transformers |
| Models | paraphrase-xlm-r-multilingual-v1 (embeddings); mDeBERTa-v3-base-mnli-xnli (inference); spaCy en_core_web_sm (parsing) |
| Search | FAISS index of Wikipedia passages; live Wikipedia search |
| Structured data | Wikidata; World Bank Open Data; WHO Global Health Observatory; Google Fact Check Tools API |
| Web | FastAPI, Uvicorn, SQLite, React, Vite, Tailwind CSS |

## 5.2 Implementation of the Modules

A request passes through the modules of Table 4.1 in the order of Fig. 4.1. Most of the waiting time comes from web look-ups rather than from the models, so the implementation concentrates on them. Up to six claims are looked up at the same time; each external service is paced separately, so that Wikipedia, Wikidata and the World Bank do not wait for one another; recent Wikidata and Wikipedia searches are remembered for a day; and a slow statistics service is abandoned after 8 seconds and one retry. None of these changes alters a verdict: the 13 claims of three test texts received the same verdicts before and after.

For demonstrations, a single server process (serve.py) delivers both the website and the API, and a secure public https link to the laptop is created with a Cloudflare tunnel. The application is also packaged as a container for cloud hosting. The code base has 265 automated tests.

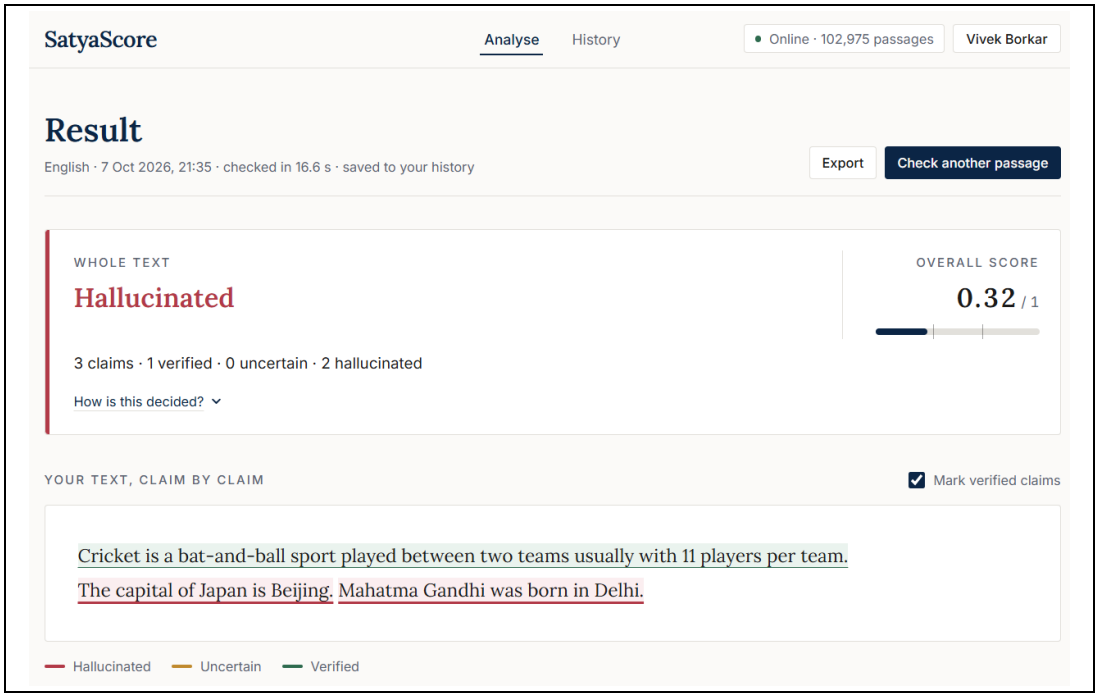
## 5.3 Web Application

A user pastes or uploads an answer, leaves the language on automatic detection (Devanagari script means Hindi) or chooses it, and starts the check. The result page shows the whole-text verdict and overall score, the text with each claim underlined in the colour of its verdict, and a card for every claim with its source, evidence and,

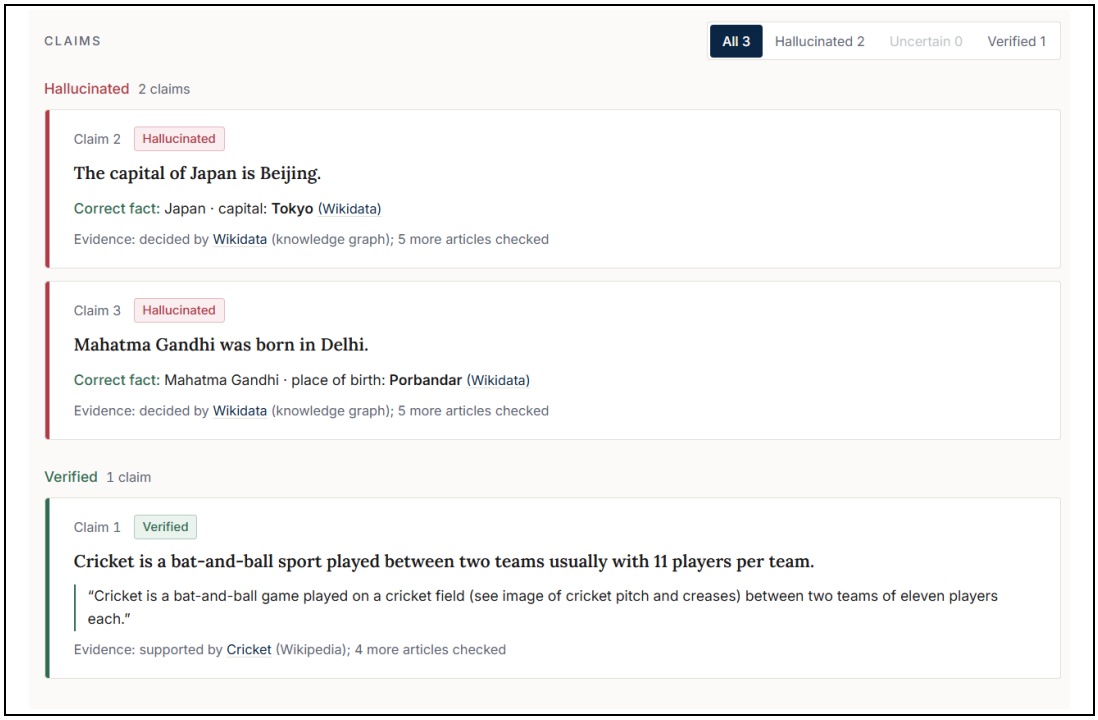
where available, the correct fact. Signed-in users keep a history of their checks and can export results as CSV or JSON. Screenshots 5.1 to 5.5 were taken from the running system.




**Screenshot 5.1: Input page with an example answer**



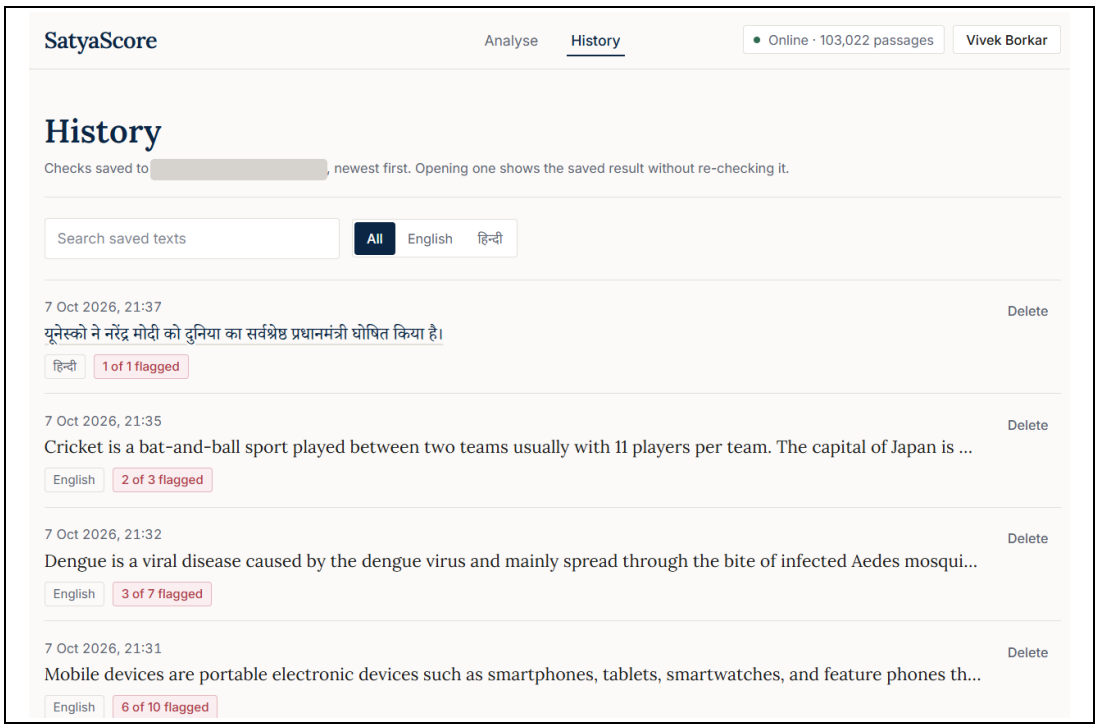

**Screenshot 5.2: Results page: whole-text verdict and highlighted claims**




**Screenshot 5.3: Claim cards with the correct fact from Wikidata**




**Screenshot 5.4: A Hindi claim decided by a published fact-check**




**Screenshot 5.5: History of saved checks (account e-mail hidden)**

Security measures: passwords are hashed with scrypt and a per-user salt; sessions use HttpOnly cookies; state-changing requests from other websites are refused by an origin check; failed sign-ins are throttled (five failures for the same account from the same address within 15 minutes block further attempts until the window clears); checks are rate-limited per user and per network address; a password change requires the current password; and a forgotten password is reset with a one-time code that is valid for 30 minutes.

### 5.4 Datasets and Annotation

Table 5.2 lists the data used for evaluation. The main set consists of claims extracted from answers that the Llama 3.3 (70B) model [2] gave to questions in five domains: government, healthcare, science, sports and history.

**Table 5.2: Datasets**

| Dataset | Source | Size | Used for |
|---|---|---|---|
| Labelled chatbot claims | Llama 3.3 (70B) answers, five domains | 456 claims (266 English, 190 Hindi); 418 well-formed | Weight training and main evaluation |
| Train / test split | 418 well-formed claims, grouped by source question; test fold fixed in advance | 334 / 84 claims; binary sets 303 (258 true, 45 hallucinated) / 76 (66 true, 10 hallucinated) | Main evaluation |

| Dataset | Source | Size | Used for |
|---|---|---|---|
| Relational claims | Answers from gpt-oss-120b and qwen3.8-27b | 45 labelled by two annotators; 40 where they agreed (25 true, 15 hallucinated); English | Knowledge-graph study |
| Unseen true sentences | Wikipedia sentences not used in development | 99 claims and 128 claims (two samples) | False-alarm tests |
| Statistics test | True and altered figures for 40 randomly chosen countries | 137 true and 137 altered claims | Statistics check |
| Fact-check test | Known rumours and true statements | 40 claims (25 rumours, 15 true) | Fact-check matching |
| Pronoun test | Held-out Wikipedia introductions | 400 introductions | Pronoun resolution |
| Common-sense set | Statements written for the error analysis | 15 statements (9 false) | Error analysis |

Annotation was done in two ways, and the difference matters. The 456 main claims were labelled by a team of four annotators, the authors, who shared the claims among themselves so that each claim was labelled by one of them. An AI system suggested a verdict for every claim, and the annotator confirmed or overrode each suggestion. Overall, 79% of the final labels agree with the suggestion. Because the suggestion was visible, these labels may be biased towards it. Two blind checks measured this. In the first, two annotators independently labelled 56 claims without seeing any suggestion; their agreement on "hallucination or not" was $\kappa = 0.53$ (88% raw agreement). In the second, 50 claims were labelled blind; where the two annotators agreed (46 claims), the existing label matched theirs in 45 cases. The relational claims were labelled by two annotators with AI assistance, and only the 40 claims on which both agreed are used.

### 5.5 Evaluation Method

For each proposed change, a decision rule stating what the change had to achieve (for example, fewer false alarms without fewer correct verdicts) was written down and committed to the project repository before the change was run. Most evaluations followed this protocol. Two analyses of how the knowledge graph should be used were exploratory: they were made after the labels had been seen and are marked as such. In four cases the final decision went against the outcome of the rule; these are reported in Section 5.8.

The measures are defined as follows. A false alarm is a true claim marked Hallucinated. A hallucination is caught when it is marked Hallucinated. A verdict is correct when it matches the label; Uncertain is never counted as correct. Claims from the same chatbot answer are always on the same side of the train/test split, so that the context of an answer cannot leak into the test.

### 5.6 Results on Real Chatbot Claims and Unseen Data

**Table 5.3: Results of the running system on chatbot claims and unseen data**

| Set | Correct | False alarms | Hallucinations caught | Hallucinations marked Verified | Uncertain |
|---|---|---|---|---|---|
| Training claims (303) | 157 | 9 of 258 | 5 of 45 | 27 of 45 | 110 |
| Test claims (76) | 38 | 3 of 66 | 0 of 10 | 8 of 10 | 27 |
| Unseen true sentences (128) | 113 (Verified) | 4 of 128 | — | — | 11 |

The system seldom calls a true statement false: 4 of 128 unseen true sentences and 3 of 66 true test claims. It is, however, weak at catching hallucinations that no structured source covers: on the training claims 27 of 45 hallucinations were marked Verified and only 5 were caught, and on the small test set none of 10 was caught. Many claims end as Uncertain.

### 5.7 Results of the Structured Sources

**Table 5.4: Results of the structured sources**

| Component | Test data | Result |
|---|---|---|
| Knowledge graph, pre-registered run | 40 relational claims (English) | 11 of 13 decisions correct: supported 8 of 8, contradicted 3 of 5 |
| Knowledge graph after a place-linking fix (exploratory) | Same 40 claims | 12 of 12 decisions correct: supported 10 of 10, contradicted 2 of 2 |
| Knowledge graph ablation, pre-registered | Same 40 claims | Inference model alone against inference plus knowledge graph: AUC 0.704 → 0.784 (difference +0.080, 95% interval +0.003 to +0.189); Hallucinated-F1 0.512 → 0.615 |
| Knowledge graph deciding when it answers (after the fix) | Same 40 claims | Correct verdicts: 14 with the inference model alone, 24 when the knowledge graph decides |

| Component | Test data | Result |
|---|---|---|
| Official statistics | 40 randomly chosen countries | 137 of 137 altered figures contradicted; of 137 true figures, 131 Verified, 0 contradicted, 6 without an answer |
| Published fact-checks | 40 claims (25 rumours, 15 true) | No wrong decision; rumours caught 3 → 10 of 25; correct verdicts 13 → 20 of 40 |
| Pronoun resolution | 400 Wikipedia introductions | Correctly resolved 93 → 277; wrongly resolved 206 → 128 |

The two incorrect decisions of the pre-registered knowledge-graph run came from place names linked to the wrong Wikidata items (for example, a town linked to a given name and a village to a monorail station). The linking was corrected after these results had been seen, which is why the 12-of-12 figure is exploratory. The rule that the knowledge graph decides when it answers was also chosen on these 40 claims after the labels were known: its score separated true from hallucinated claims slightly better than the weighted combination (AUC 0.872 against 0.848; difference +0.024, 95% interval −0.000 to +0.083). The pre-registered ablation, by contrast, was fixed before any label existed and shows that adding the knowledge graph to the inference signal improves the separation of true and hallucinated claims. The knowledge-graph results are for English claims; the Hindi templates are implemented but were not part of this test.

### 5.8 Changes Tested and Not Adopted

Table 5.5 lists the main changes and the decision taken on each. The first four rows are the cases in which the final decision went against the outcome of the rule.

**Table 5.5: Changes tested and the decisions taken**

| Change | Measured outcome | Decision |
|---|---|---|
| Evidence rule (Section 4.4.3) | False alarms on 99 unseen true statements 33 → 8; but its pre-fixed check on training AUC failed, and hallucinations caught fell (train 11 → 3 of 45, test 3 → 0 of 10, relational 12 → 2 of 15) | Adopted against the rule, to avoid calling true text false |
| Reading 10 passages instead of 3 | Correct training verdicts 130 → 170 of 303; but its pre-fixed check on false alarms failed (8 → 11 of 258) | Adopted against the rule |

| Change | Measured outcome | Decision |
|---|---|---|
| LLM judge (open mode) | Met its rule (training AUC 0.717 against 0.552); caught 18 instead of 5 of 45 hallucinations, but false alarms rose from 9 to 35 of 258 | Kept off against the rule |
| Larger inference models | The rule picked other models (English: train AUC 0.602 against 0.539, intervals overlapping; Hindi: too few hallucinations to judge) | Current model kept against the rule |
| Number guard (two versions) | Removed correct verifications (115 → 101; 109 → 103) | Not adopted |
| MedlinePlus as a second source | False alarms 9 → 11 of 258 | Not adopted |
| Strict support rule | True claims Verified 152 → 47; false alarms 9 → 11 | Not adopted |
| Wikipedia articles fetched on demand | First test: no gain (ΔAUC −0.020); second test, with the evidence rule: all three conditions met | Adopted after the second test |
| MiniCheck grounding checker (probe) | Still supported 7 of 9 false common-sense statements, as the current model did | Not pursued |

These results are informative in themselves. The method that caught the most hallucinations, an LLM judge, did so at a cost in false alarms that the project was not willing to accept, and the evidence rule bought its low false-alarm rate with fewer detections.

### 5.9 Performance

**Table 5.6: Performance before and after optimisation (laptop GPU)**

| Text | Before: first / repeated check | After: first / repeated check |
|---|---|---|
| Chatbot answer, 6 claims (English) | 27.7 s / 10.1 s | 5.1 s / 0.7 s |
| General facts, 4 claims (English) | 7.5 s / 4.4 s | 1.6 s / 0.3 s |
| Hindi answer, 3 claims | 15.9 s / 9.4 s | 3.7 s / 0.4 s |

Without a GPU, the six-claim answer took 13.7 s on first check and 0.7 s on repetition. Starting the server loads the models once, which takes about 20 seconds.

### 5.10 Discussion and Limitations

**Comparison with existing methods.** The systems in Table 2.1 were evaluated on other data and languages, so their published scores cannot be compared directly with

these results. Qualitatively, Satya-Score is the only one among them that checks Hindi claims against structured sources and states the correct fact. On this project's data, a purpose-built grounding checker and larger inference models did not improve on the model in use, and an LLM judge traded many more false alarms for its extra detections.

**Error analysis.** Of the 45 hallucinations in the training split, about 31 turned out to come from an earlier version of the claim extractor rather than from the chatbot. Most of the remaining ones are misconceptions or over-generalisations that no structured source records; several were marked Verified because a retrieved passage was merely about the same topic, the word-overlap weakness described by McCoy et al. [17]. On 15 common-sense statements written for this analysis, all 9 false ones were marked Verified.

**Limitations:**

- Hallucinations outside the coverage of structured sources are usually missed.
- The labelled data are small (456 main and 40 relational claims) and contain few Hindi hallucinations (9).
- Each main claim was labelled once, by one of the four annotators, with the AI suggestion visible; agreement between independent annotators was moderate ($\kappa = 0.53$).
- The knowledge-graph rules were chosen in exploratory analyses on the same 40 claims on which they were measured.
- Live look-ups need internet access, and the WHO data may be used for non-commercial purposes only.

# CHAPTER 6
# CONCLUSION

## 6.1 Conclusion

Satya-Score checks English and Hindi answers one claim at a time and supports every verdict with a source that the reader can open. Sending each claim first to official statistics, the Wikidata knowledge graph or a published fact-check made the system precise wherever such a source applies, and the conservative evidence rule kept false alarms on true text low. The same evaluation shows the price of that caution: most hallucinations that no structured source covers are not caught. Testing each change against a rule fixed in advance made clear which attractive ideas did not help, and the cases in which the project decided against a rule are reported openly. In one sentence, the contribution is an explainable bilingual claim checker that combines published fact-checks, official statistics, a knowledge graph and encyclopaedic text while keeping false alarms low.

## 6.2 Future Scope

- Catching misconceptions that no structured source covers without adding false alarms, for example by letting an LLM propose suspect claims that a structured source must then confirm.
- Extending the system to Marathi with MuRIL, IndicSBERT and IndicXNLI.
- A larger, multi-domain Hindi evaluation set, labelled blind by at least two annotators.
- Further official sources, such as Indian government open-data portals.

# REFERENCES


[1] OpenAI et al., "GPT-4 technical report," arXiv:2303.08774, 2023, doi: 10.48550/arXiv.2303.08774.

[2] A. Grattafiori et al., "The Llama 3 herd of models," arXiv:2407.21783, 2024, doi: 10.48550/arXiv.2407.21783.

[3] Z. Ji et al., "Survey of hallucination in natural language generation," ACM Comput. Surv., vol. 55, no. 12, 2023, doi: 10.1145/3571730.

[4] L. Huang et al., "A survey on hallucination in large language models: Principles, taxonomy, challenges, and open questions," ACM Trans. Inf. Syst., vol. 43, no. 2, 2025, doi: 10.1145/3703155.

[5] S. Min et al., "FActScore: Fine-grained atomic evaluation of factual precision in long form text generation," in Proc. EMNLP, 2023, doi: 10.48550/arXiv.2305.14251.

[6] J. Thorne, A. Vlachos, C. Christodoulopoulos, and A. Mittal, "FEVER: A large-scale dataset for fact extraction and VERification," in Proc. NAACL-HLT, 2018, doi: 10.18653/v1/N18-1074.

[7] O. Honovich et al., "TRUE: Re-evaluating factual consistency evaluation," in Proc. NAACL-HLT, 2022, doi: 10.18653/v1/2022.naacl-main.287.

[8] Y. Zha, Y. Yang, R. Li, and Z. Hu, "AlignScore: Evaluating factual consistency with a unified alignment function," in Proc. ACL, 2023, doi: 10.18653/v1/2023.acl-long.634.

[9] S. Lin, J. Hilton, and O. Evans, "TruthfulQA: Measuring how models mimic human falsehoods," in Proc. ACL, 2022, doi: 10.18653/v1/2022.acl-long.229.

[10] J. Li, X. Cheng, X. Zhao, J.-Y. Nie, and J.-R. Wen, "HaluEval: A large-scale hallucination evaluation benchmark for large language models," in Proc. EMNLP, 2023, doi: 10.18653/v1/2023.emnlp-main.397.

[11] J. Wei et al., "Long-form factuality in large language models," arXiv:2403.18802, 2024, doi: 10.48550/arXiv.2403.18802.

[12] P. Manakul, A. Liusie, and M. J. F. Gales, "SelfCheckGPT: Zero-resource black-box hallucination detection for generative large language models," in Proc. EMNLP, 2023, doi: 10.48550/arXiv.2303.08896.

[13] X. Hu et al., "Knowledge-centric hallucination detection," in Proc. EMNLP, 2024, doi: 10.18653/v1/2024.emnlp-main.395.

[14] Z. Ji et al., "ANAH: Analytical annotation of hallucinations in large language models," in Proc. ACL, 2024, doi: 10.18653/v1/2024.acl-long.442.

[15] A. Mishra, A. Asai, V. Balachandran, Y. Wang, G. Neubig, Y. Tsvetkov, and H. Hajishirzi, "Fine-grained hallucination detection and editing for language models," in Proc. COLM, 2024, doi: 10.48550/arXiv.2401.06855.

[16] W. Su, C. Wang, Q. Ai, Y. Hu, Z. Wu, Y. Zhou, and Y. Liu, "Unsupervised real-time hallucination detection based on the internal states of large language models," in Findings of ACL, 2024, doi: 10.18653/v1/2024.findings-acl.854.

[17] R. T. McCoy, E. Pavlick, and T. Linzen, "Right for the wrong reasons: Diagnosing syntactic heuristics in natural language inference," in Proc. ACL, 2019, doi: 10.18653/v1/P19-1334.

[18] L. Tang, P. Laban, and G. Durrett, "MiniCheck: Efficient fact-checking of LLMs on grounding documents," in Proc. EMNLP, 2024, doi: 10.48550/arXiv.2404.10774.

[19] P. Lewis et al., "Retrieval-augmented generation for knowledge-intensive NLP tasks," in Proc. NeurIPS, 2020, doi: 10.48550/arXiv.2005.11401.

[20] R. Vázquez et al., "SemEval-2025 Task 3: Mu-SHROOM, the multilingual shared task on hallucinations and related observable overgeneration mistakes," in Proc. SemEval, 2025, arXiv:2504.11975, doi: 10.48550/arXiv.2504.11975.

[21] B. Hikal, A. Nasreldin, and A. Hamdi, "MSA at SemEval-2025 Task 3: High quality weak labeling and LLM ensemble verification for multilingual hallucination detection," in Proc. SemEval, 2025. [Online]. Available: https://aclanthology.org/2025.semeval-1.131/

[22] M. Bala et al., "HausaNLP at SemEval-2025 Task 3: Towards a fine-grained model-aware hallucination detection," in Proc. SemEval, 2025. [Online]. Available: https://aclanthology.org/2025.semeval-1.227/

[23] P. Elchafei and M. Abu-Elkheir, "Hallucination Detectives at SemEval-2025 Task 3: Span-level hallucination detection for LLM-generated answers," in Proc. SemEval, 2025. [Online]. Available: https://aclanthology.org/2025.semeval-1.84/

[24] A. Sinha et al., "SHROOM-CAP: Shared task on hallucinations and related observable overgeneration mistakes in crosslingual analyses of publications," in Proc. CHOMPS, 2025, doi: 10.18653/v1/2025.chomps-main.7.

[25] H. Terdalkar, K. Bhojani, A. Dongare, and O. A. Behera, "BHRAM-IL: A benchmark for hallucination recognition and assessment in multiple Indian languages," in Proc. BHASHA Workshop, 2025, doi: 10.18653/v1/2025.bhasha-1.9.

[26] A. Conneau et al., "Unsupervised cross-lingual representation learning at scale," in Proc. ACL, 2020, doi: 10.18653/v1/2020.acl-main.747.

[27] S. Khanuja et al., "MuRIL: Multilingual representations for Indian languages," arXiv:2103.10730, 2021, doi: 10.48550/arXiv.2103.10730.

[28] D. Kakwani et al., "IndicNLPSuite: Monolingual corpora, evaluation benchmarks and pre-trained multilingual language models for Indian languages," in Findings of EMNLP, 2020, doi: 10.18653/v1/2020.findings-emnlp.445.

[29] S. Deode et al., "L3Cube-IndicSBERT: A simple approach for learning cross-lingual sentence representations using multilingual BERT," arXiv:2304.11434, 2023, doi: 10.48550/arXiv.2304.11434.

[30] D. Aggarwal, V. Gupta, and A. Kunchukuttan, "IndicXNLI: Evaluating multilingual inference for Indian languages," arXiv:2204.08776, 2022, doi: 10.48550/arXiv.2204.08776.

[31] A. Williams, N. Nangia, and S. R. Bowman, "A broad-coverage challenge corpus for sentence understanding through inference," in Proc. NAACL-HLT, 2018, doi: 10.18653/v1/N18-1101.

[32] A. Conneau et al., "XNLI: Evaluating cross-lingual sentence representations," in Proc. EMNLP, 2018, doi: 10.18653/v1/D18-1269.

[33] D. Vrandečić and M. Krötzsch, "Wikidata: A free collaborative knowledgebase," Commun. ACM, vol. 57, no. 10, pp. 78–85, 2014, doi: 10.1145/2629489.

[34] J. Kim, S. Park, Y. Kwon, Y. Jo, J. Thorne, and E. Choi, "FactKG: Fact verification via reasoning on knowledge graphs," in Proc. ACL, 2023, doi: 10.18653/v1/2023.acl-long.895.

[35] X. Guan et al., "Mitigating large language model hallucinations via autonomous knowledge graph-based retrofitting," in Proc. AAAI, vol. 38, no. 16, pp. 18126–18134, 2024, doi: 10.1609/aaai.v38i16.29770.

[36] K. Lavrinovics, R. Biswas, K. Hose, and J. Bjerva, "MultiHal: Multilingual dataset for knowledge-graph grounded evaluation of LLM hallucinations," arXiv:2505.14101, 2025, doi: 10.48550/arXiv.2505.14101.

[37] G. Agrawal, T. Kumarage, Z. Alghamdi, and H. Liu, "Can knowledge graphs reduce hallucinations in LLMs? A survey," arXiv:2311.07914, 2023, doi: 10.48550/arXiv.2311.07914.

[38] K. Lavrinovics, R. Biswas, J. Bjerva, and K. Hose, "Knowledge graphs, large language models, and hallucinations: An NLP perspective," J. Web Semantics, vol. 85, art. 100844, 2025, doi: 10.1016/j.websem.2024.100844.

[39] N. Reimers and I. Gurevych, "Sentence-BERT: Sentence embeddings using Siamese BERT-networks," in Proc. EMNLP-IJCNLP, 2019, doi: 10.18653/v1/D19-1410.

[40] N. Reimers and I. Gurevych, "Making monolingual sentence embeddings multilingual using knowledge distillation," in Proc. EMNLP, 2020, doi: 10.48550/arXiv.2004.09813.

[41] J. Johnson, M. Douze, and H. Jégou, "Billion-scale similarity search with GPUs," IEEE Trans. Big Data, vol. 7, no. 3, pp. 535–547, 2021, doi: 10.1109/TBDATA.2019.2921572.

[42] P. He, J. Gao, and W. Chen, "DeBERTaV3: Improving DeBERTa using ELECTRA-style pre-training with gradient-disentangled embedding sharing," in Proc. ICLR, 2023, doi: 10.48550/arXiv.2111.09543.

[43] M. Honnibal, I. Montani, S. Van Landeghem, and A. Boyd, "spaCy: Industrial-strength natural language processing in Python," Zenodo, 2020, doi: 10.5281/zenodo.1212303.

[44] A. Kunchukuttan, "The IndicNLP Library," 2020. [Online]. Available: https://github.com/anoopkunchukuttan/indic_nlp_library

[45] The World Bank, "World Bank Open Data." [Online]. Available: https://data.worldbank.org

[46] World Health Organization, "The Global Health Observatory." [Online]. Available: https://www.who.int/data/gho

[47] Google, "Fact Check Tools API." [Online]. Available: https://developers.google.com/fact-check/tools/api

[48] A. Paszke et al., "PyTorch: An imperative style, high-performance deep learning library," in Proc. NeurIPS, 2019, arXiv:1912.01703.

[49] T. Wolf et al., “Transformers: State-of-the-art natural language processing,” in Proc. EMNLP: System Demonstrations, 2020, doi: 10.18653/v1/2020.emnlp-demos.6.

# LIST OF PUBLISHED PAPERS AND IPR FILED

Published papers: none at the time of writing this report.

| Sr. No. | Title | Type | Status / Number |
|---|---|---|---|
| 1 | Satya-Score: An AI Hallucination Detection & Scoring Framework | Copyright (to be filed through the institute) | [Diary / application number, once filed] |

# RESUME

[Attach the resume of each group member: the individual copy contains the student's own resume; the department copy contains the resumes of all group members.]